

3

Mecanismos adicionais de sincronização

3.1 Introdução

Sincronização refere-se a qualquer mecanismo que permite controlar a ordem de execução de operações em diferentes *threads* Andrews 1991. Em programas concorrentes de memória compartilhada as formas principais de sincronização são:

- Exclusão mútua
- Sincronização por condição
- Barreiras
- Semáforos

A **exclusão mútua** é normalmente realizada com uso de ‘mutexes’, que garantem que apenas uma *thread* acesse, por vez, uma seção crítica de código que manipula dados compartilhados, evitando corrupção de estado e condições de corrida. Enquanto uma *thread* está na região crítica, as demais esperam até que ela saia.

Uma outra opção é usar **variáveis de condição** para fazer uma *thread* esperar até que uma certa condição lógica torne-se verdadeira (por exemplo, *buffer* não vazio para um consumidor). Quando a condição é satisfeita, outra *thread* “acorda” a(s) thread(s) em espera, permitindo coordenação mais fina do que simples exclusão mútua.

A **barreira** é um ponto de sincronização, que funciona como um ponto de encontro, onde várias *threads* co-ordenadas pausam a execução até que todas cheguem àquele ponto. Só então o conjunto de *threads* é liberado para continuar, o que é útil em aplicações paralelas em etapas (por exemplo, cada etapa de um cálculo iterativo).

Os **semáforos** são variáveis inteiras utilizadas como contadores e acessados com operações atômicas *wait* (P) e *signal* (V) usadas para controlar o acesso a recursos compartilhados. Não faz parte do padrão pthreads, mas são bastante úteis em diversas situações. Um semáforo pode ser binário (imitando um *mutex* de exclusão mútua) ou contador (permitindo até *k* processos acessando um recurso simultaneamente).

3.2 Variáveis de Condição

As variáveis de condição são uma forma de implementação de sincronização por meio de variáveis especiais que permitem que as *threads* esperem (bloqueando-se) até que sejam sinalizadas (avisadas) por outra *thread* que a condição lógica foi atendida. Associadas a essas variáveis de condição, temos normalmente três operações principais:

- WAIT(condvar): bloqueia a *thread* na fila da variável de condição;
- SIGNAL(condvar): desbloqueia uma *thread* na fila de espera da variável de condição;
- BROADCAST(condvar): desbloqueia todas as *threads* na fila da variável de condição.

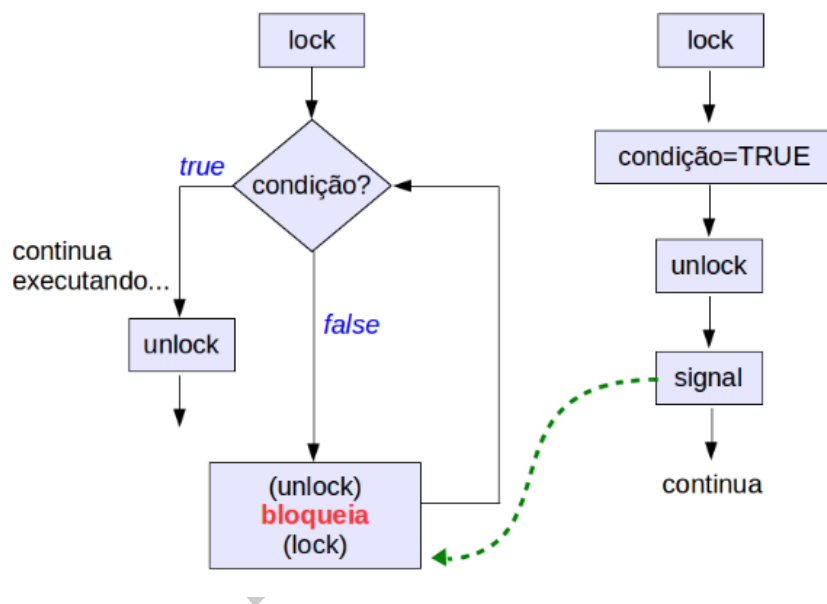


Figura 3.1: Um fluxo avalia o estado da aplicação, se a condição lógica estiver satisfeita, ele segue sem bloqueio, caso contrário, ele se bloqueia na fila da variável de condição. Um outro fluxo da aplicação deverá sinalizá-lo quando a condição lógica se tornar verdadeira.

Uma variável de condição deve sempre ser usada em conjunto com um *mutex*. Este protege a verificação da condição lógica da aplicação, sendo automaticamente liberado durante o *wait* e readquirido quando a *thread* retoma a execução do ponto de bloqueio. A Figura 3.1 ilustra esse funcionamento. O uso do *mutex*:

- Protege a verificação da condição;
- É liberado automaticamente durante o *wait*;
- É readquirido ao retomar execução.

3.2.1 Funções da biblioteca Pthreads

A biblioteca Pthreads define um tipo especial chamado `pthread_cond_t` que pode ser utilizado com as seguintes rotinas:

- **`int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)`**

Esta função bloqueia a *thread* na condição (condvar) (deve ser chamada com *mutex* alocado para a *thread* e depois de finalizado deve desalocar o *mutex*);

- **`int pthread_cond_signal(pthread_cond_t *cond)`** A função desbloqueia uma *thread* esperando pela condição (cond);

- **`int pthread_cond_broadcast(pthread_cond_t *cond)`**

Essa função é utilizada no lugar de SIGNAL quando todas as *threads* na fila da condição podem ser desbloqueadas;

- **`int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *attr)`**

Inicializa a variável de condição referenciada por 'cond' com os atributos referenciados por 'attr'. Se 'attr' for 'NULL', são utilizados os atributos padrão da variável de condição.

- **`int pthread_cond_destroy(pthread_cond_t *cond)`**

Essa função libera a variável de condição especificada por 'cond'.

3.2.2 Exemplo de uso de variáveis condicionais

A seguir um exemplo de produtor/consumidor utilizando variáveis condicionais.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int dado_pronto = 0;
int dado = 0;

void *produtor(void *arg) {
    sleep(2); // simula trabalho

    pthread_mutex_lock(&mutex);
    dado = 42;
```

```

    dado_pronto = 1;
    printf("Produtor: dado pronto = %d\n", dado);
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);

    return NULL;
}

void *consumidor(void *arg) {
    pthread_mutex_lock(&mutex);

    while (dado_pronto == 0) {
        printf("Consumidor: aguardando...\n");
        pthread_cond_wait(&cond, &mutex);
    }

    printf("Consumidor: recebeu dado = %d\n", dado);
    pthread_mutex_unlock(&mutex);

    return NULL;
}

int main(void) {
    pthread_t t_prod, t_cons;

    pthread_create(&t_cons, NULL, consumidor, NULL);
    pthread_create(&t_prod, NULL, produtor, NULL);

    pthread_join(t_prod, NULL);
    pthread_join(t_cons, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);

    return 0;
}

```

3.3 Sincronização por Barreira

Um tipo particular de sincronização por condição aparece em problemas que requerem que todos os fluxos de execução (ou um grupo deles) sincronizem suas ações em determinados pontos da execução do algoritmo. Essa sincronização é chamada de sincronização por barreira porque funciona exatamente como uma “barreira” (ou bloqueio coletivo) que faz com que todos as *threads* aguardem pelas demais até que todas tenham chegado a esse ponto (ou passo) do algoritmo. A Figura 3.2 ilustra essa situação.

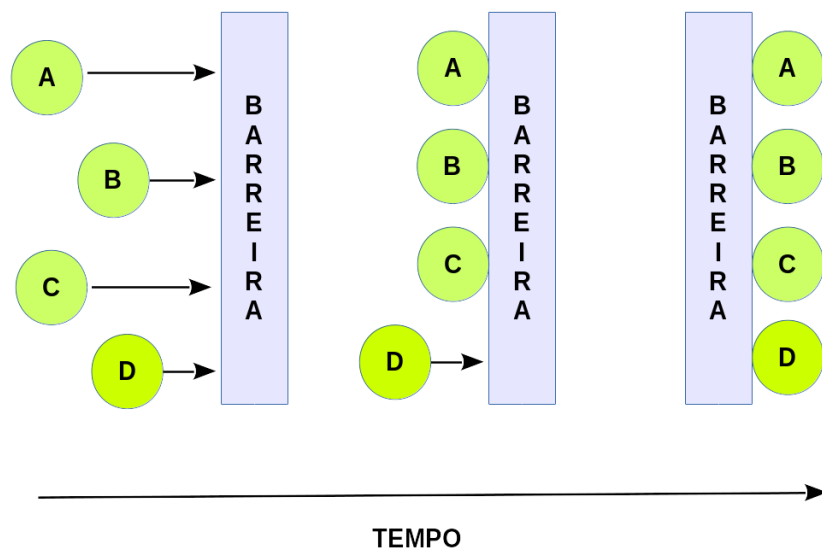


Figura 3.2: Barreira

3.3.1 Funções da biblioteca Pthreads

A biblioteca *pthread*s oferece uma rotina que implementa essa operação, onde o número de *threads* participantes da operação deve ser passado como parâmetro durante a sua inicialização.

- **`int pthread_barrier_init(pthread_barrier_t *restrict barrier, const pthread_barrierattr_t *restrict attr, unsigned count)`**

Esta função deve alocar quaisquer recursos necessários para utilizar a barreira referenciada por 'barrier' e deve inicializá-la com os atributos referenciados por 'attr'. Se 'attr' for 'NULL', serão utilizados os atributos padrão da barreira; o efeito é o mesmo que passar o endereço de um objeto de atributos de barreira padrão. O parâmetro 'count' indica o número de *threads* participantes da barreira.

- **`int pthread_barrier_wait(pthread_barrier_t *barrier)`**

A função deve sincronizar as *threads* participantes na barreira referenciada por 'barrier'. A *thread* chamadora deve permanecer bloqueada até que o número necessário de *threads* tenha chamado 'pthread_barrier_wait()' com esta barreira específica.

- **`int pthread_barrier_destroy(pthread_barrier_t *barrier)`**

Esta função deve destruir a barreira referenciada por 'barrier' e liberar quaisquer recursos utilizados por essa barreira.

3.3.2 Exemplo de uso de barreira

```
#include <stdio.h>
#include <pthread.h>

#define THREAD_NUMS 4
pthread_barrier_t barrier;

void *t0(void *param)
{
    pthread_barrier_wait(&barrier);
    printf("t0 pronta\n");
}
void *t1(void *param)
{
    pthread_barrier_wait(&barrier);
    printf("t1 pronta.\n");
}
void *t2(void *param)
{
    pthread_barrier_wait(&barrier);
    printf("t2 pronta.\n");
}

int main(void)
{
    pthread_t t[3];
    // Inicia a barreira e define o numero de threads participantes
    pthread_barrier_init(&barrier, NULL, THREAD_NUMS);

    pthread_create(&t[0], NULL, t0, NULL);
    pthread_create(&t[1], NULL, t1, NULL);
    pthread_create(&t[2], NULL, t2, NULL);

    pthread_barrier_wait(&barrier);
    printf("todas as threads prontas, pode seguir!\n");

    pthread_barrier_destroy(&barrier);
}
```

3.3.3 Aplicação: Método de Jacobi

Os métodos iterativos para solução de equações lineares são exemplos de problemas que quando paralelizados normalmente requerem o uso da sincronização por barreira. Esses problemas possuem passos ou etapas bem definidas onde ocorre uma verificação para decidir se o programa deve continuar executando ou se a solução encontrada até esse passo já é satisfatória. Quando paralelizamos esse tipo de problema, os pontos de verificação que antecedem uma nova etapa do algoritmo precisam ser precedidos por uma barreira para

garantir que todas as *threads* cheguem a esse ponto do código antes de realizar a próxima verificação, de forma que os resultados dos processamentos realizados na etapa atual sejam corretamente avaliados.

Tomemos o método iterativo de Jacobi como exemplo. Uma alternativa simples para particionar esse problema em subproblemas que podem ser executados ao mesmo tempo é atribuir para cada fluxo de execução o cálculo de uma das incógnitas do sistema. Como a cada passo é necessário garantir que todas as incógnitas foram calculadas para então iniciar o próximo passo, os fluxos de execução precisam aguardar o resultado dessa computação para então executarem a próxima iteração ou finalizarem.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <math.h>

#define N 4                // tamanho do sistema
#define MAX_IT 1000
#define TOL 1e-6
#define NTHREADS 2

double A[N][N] = {
    {10, -1,  2,  0},
    {-1, 11, -1,  3},
    { 2, -1, 10, -1},
    { 0,  3, -1,  8}
};

double b[N] = {6, 25, -11, 15};

double x[N];        // solução atual
double x_new[N];    // próxima iteração

pthread_barrier_t barrier;

typedef struct {
    int id;
    int start;
    int end;
} thread_data_t;

void* jacobi_thread(void* arg) {
    thread_data_t* data = (thread_data_t*) arg;
    int i, j, k;

    for (k = 0; k < MAX_IT; k++) {

        // Etapa 1: calcular x_new
        for (i = data->start; i < data->end; i++) {
```

```

        double sigma = 0.0;
        for (j = 0; j < N; j++) {
            if (j != i) {
                sigma += A[i][j] * x[j];
            }
        }
        x_new[i] = (b[i] - sigma) / A[i][i];
    }

    // Barreira: garantir que todos terminaram cálculo
    pthread_barrier_wait(&barrier);

    // Etapa 2: copiar x_new para x
    for (i = data->start; i < data->end; i++) {
        x[i] = x_new[i];
    }

    // Barreira: garantir atualização completa
    pthread_barrier_wait(&barrier);

    // (Opcional) critério de parada simples (apenas thread 0 verifica)
    if (data->id == 0) {
        double erro = 0.0;
        for (i = 0; i < N; i++) {
            erro += fabs(x_new[i] - x[i]);
        }
        if (erro < TOL) {
            printf("Convergiu em %d iteracoes\n", k);
            exit(0);
        }
    }
}

pthread_exit(NULL);
}

int main() {
    pthread_t threads[NTHREADS];
    thread_data_t data[NTHREADS];

    // Inicialização
    for (int i = 0; i < N; i++) {
        x[i] = 0.0;
    }

    pthread_barrier_init(&barrier, NULL, NTHREADS);

    int chunk = N / NTHREADS;

    // Criação das threads
    for (int t = 0; t < NTHREADS; t++) {

```



```

    data[t].id = t;
    data[t].start = t * chunk;
    data[t].end = (t == NTHREADS - 1) ? N : (t + 1) * chunk;

    pthread_create(&threads[t], NULL, jacobi_thread, &data[t]);
}

// Espera threads
for (int t = 0; t < NTHREADS; t++) {
    pthread_join(threads[t], NULL);
}

pthread_barrier_destroy(&barrier);

// Resultado final
printf("Solucao aproximada:\n");
for (int i = 0; i < N; i++) {
    printf("x[%d] = %f\n", i, x[i]);
}

return 0;
}

```

3.4 Semáforos

Os semáforos são ferramentas de sincronização usadas em sistemas operacionais para controlar o acesso a recursos compartilhados em ambientes com múltiplos processos ou múltiplas *threads*. Eles são variáveis inteiras que coordenam a execução por meio de operações atômicas, como `wait()` e `signal()`. Dessa forma, ajudam a evitar condições de corrida e a garantir a coordenação correta entre processos.

Principais funções

Os semáforos controlam a entrada em seções críticas, mantêm um contador representando a quantidade disponível de instâncias do recurso, garantem exclusão mútua entre processos ou *threads*, bloqueiam e acordam esses fluxos de execução durante a programação concorrente, sendo amplamente utilizados nesse contexto.

Características dos semáforos

A exclusão mútua impede que mais de um processo ou *thread* acesse simultaneamente um recurso compartilhado, preservando a consistência dos dados e evitando condições de corrida. Além disso, esse mecanismo é essencial para a sincronização entre tarefas concorrentes, pois permite coordenar a ordem de execução e controlar o momento em que cada fluxo de execução pode prosseguir. Em sistemas com recursos finitos, como impressoras, dispositivos de E/S e conexões de banco de dados, ele também atua no gerenciamento de recursos, limitando o acesso

ao número de instâncias disponíveis. Um caso clássico é o problema leitor-escritor, no qual múltiplos leitores podem acessar os dados ao mesmo tempo, mas escritores devem aguardar enquanto houver leitores ativos, garantindo integridade e desempenho equilibrados. Em alguns cenários, o uso cuidadoso desse mecanismo também contribui para prevenir *deadlocks*, desde que a alocação dos recursos siga uma ordem global consistente entre todos os processos ou *threads*.

Limitações dos semáforos

Embora fundamentais na computação concorrente, o uso de semáforos apresenta desafios críticos que demandam atenção rigorosa do desenvolvedor. A inversão de prioridade é um risco relevante, onde um processo de baixa prioridade, ao possuir um semáforo, acaba por bloquear indevidamente uma tarefa de alta prioridade. Além disso, falhas no gerenciamento complexo — que exige um controle preciso entre chamadas de ‘wait’ e ‘signal’ — podem levar a *deadlocks*, situações em que múltiplos processos aguardam uns pelos outros em ciclos de dependência sem solução. Adicionalmente, implementações mal otimizadas podem resultar em "espera ocupada" (*busy waiting*), onde processos verificam repetidamente o estado do semáforo, gerando desperdício desnecessário de ciclos de processador que poderiam ser dedicados a tarefas produtivas.

Funcionamento dos semáforos

Um semáforo em sistemas operacionais usa duas operações atômicas principais:

1. **wait(S)**: também chamada de operação de espera ou aquisição.
 - Decrementa o valor do semáforo;
 - Se o valor ficar menor que zero, o processo entra em espera até que o recurso esteja disponível;
 - É usada para adquirir um recurso.
2. **signal(S)**: Também chamada de operação de liberação.
 - Incrementa o valor do semáforo;
 - Se houver processos ou *threads* esperando, um deles é acordado;
 - É usada para liberar um recurso.

3.4.1 Exemplo de uso

Considere dois processos, ‘P1’ e ‘P2’, compartilhando um semáforo ‘S’ inicializado com valor ‘1’, conforme ilustrado da Figura 3.3:

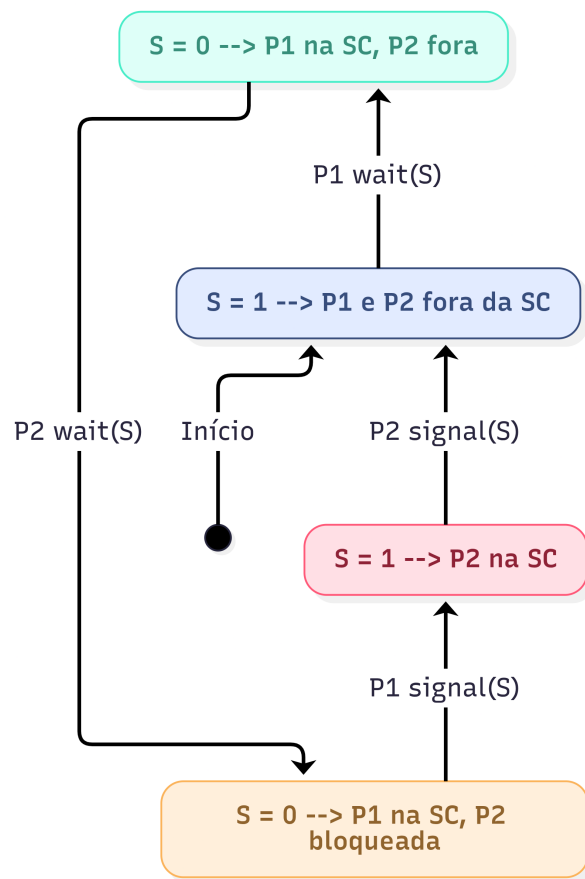


Figura 3.3: Exemplo de uso de semáforos

- **Estado 1:** 'P1' e 'P2' estão fora da seção crítica, e 'S = 1'.
- **Estado 2:** 'P1' entra na seção crítica e executa 'wait(S)', então 'S = 0'. 'P2' continua fora da seção crítica.
- **Estado 3:** Se 'P2' tentar entrar nesse momento, ele não consegue, porque 'S = 0'. Ele precisa esperar.
- **Estado 4:** Quando 'P1' termina, executa 'signal(S)', e o valor volta para 'S = 1'. Agora 'P2' pode entrar na seção crítica e fazer 'S = 0' novamente.

Esse mecanismo garante exclusão mútua, assegurando que apenas um processo acesse o recurso compartilhado por vez.

3.4.2 Tipos de semáforos

Os semáforos são normalmente classificados em dois tipos principais:

- **Semáforo de contagem:** O semáforo de contagem pode assumir valores de '0' até qualquer inteiro positivo, permitindo gerenciar o acesso simultâneo a múltiplas instâncias

de um recurso limitado. Diferente do semáforo binário, ele controla quando há N recursos disponíveis, autorizando até N *threads* ou processos concorrentes a utilizar este recurso. Podemos dar como exemplo o controle de acesso a 5 impressoras ou 3 conexões de banco de dados.

Semáforo binário: O semáforo binário só pode assumir os valores ‘0’ e ‘1’, sendo utilizado principalmente para exclusão mútua entre processos. Ele garante que apenas um processo entre na seção crítica por vez, sendo conceitualmente parecido com um *mutex*.

Além desta classificação, os semáforos podem ser separados em duas outras categorias.

- **Semáforos não nomeados:** São criados em memória dentro de um processo, usando **`sem_init(&sem, pshared, value)`**. São tipicamente usados para sincronizar *threads* do mesmo processo ou entre processos que compartilham uma memória (por exemplo, mmap ou segmentos de memória compartilhada). São destruídos com **`sem_destroy(&sem)`** quando o processo decide que não serão mais usados. Não têm ‘nome’ visível no sistema; o acesso é feito apenas pelo ponteiro ‘`sem_t *`’.
- **Semáforos nomeados:** São associados a um nome global (ex.: `/meu_semaforo`) e criados/abertos com **`sem_open (“/meu_semaforo”, O_CREAT, mode, value)`**. Podem ser acessados por processos diferentes, mesmo que não compartilhem diretamente a memória — cada processo abre o semáforo pelo nome. São fechados com **`sem_close(&sem)`** e a função **`sem_unlink (“/meu_semaforo”)`** é utilizada para remove-los quando não forem mais necessários. Comportam-se como “recursos de sistema global” e permanecem presentes até que sejam explicitamente removidos.

3.4.3 Funções da biblioteca Pthreads

Em C com semáforos POSIX (biblioteca ‘`semaphore.h`’), A seguir, elencamos as principais funções de criação e manipulação de semáforos, mas primeiramente apresentamos como incluir e declarar um semáforo.

```
#include <semaphore.h>
sem_t sem;
```

E a seguir cada uma das funções:

- **`int sem_init(sem_t *sem, int pshared, unsigned int value)`**

Inicializa um semáforo **não nomeado**. Onde:

- ‘`sem`’: ponteiro para a variável do tipo ‘`sem_t`’

- ‘pshared = 0’: o semáforo é usado apenas entre *threads* do mesmo processo.
- ‘value’: valor inicial do semáforo (0, 1, 3, etc., para semáforo de contagem).

- **sem_t *sem_open(const char *name, int oflag, ...);**

Cria ou abre um **semáforo nomeado** (útil entre processos distintos).

- **int sem_wait(sem_t *sem);**

Decrementa o valor do semáforo; se o valor é 0, bloqueia até que outro processo/thread chame a rotina ‘sem_post()’.

- **int sem_post(sem_t *sem);**

Incrementa o valor do semáforo; e desperta uma das *threads* bloqueadas em ‘sem_wait()’, se houver.

- **int sem_destroy(sem_t *sem);**

Destrói um semáforo **não nomeado**, liberando recursos internos. Deve ser chamado quando o semáforo não for mais usado.

- **int sem_close(sem_t *sem);**

Fecha um **semáforo nomeado** por um processo.

- **int sem_unlink(const char *name);**

Remove o semáforo nomeado do sistema quando todos os processos encerraram o uso.

3.4.4 Exemplo de uso de semáforos

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define NUM_THREADS 6
#define VAGAS 3

sem_t semaforo;

void* carro(void *arg) {
    int id = *(int*)arg;

    printf("Carro %d: tentando entrar...\n", id);
    sem_wait(&semaforo);

    printf("Carro %d: entrou na vaga.\n", id);
    sleep(2); // simula tempo usando o recurso
```

```

    printf("Carro %d: saiu da vaga.\n", id);

    sem_post(&semaforo);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];

    sem_init(&semaforo, 0, VAGAS);

    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, carro, &ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    sem_destroy(&semaforo);

    return 0;
}

```

Exemplo de uso de Semáforo

3.5 Exercícios

1. O que é sincronização em programas concorrentes e qual a sua finalidade principal?
2. Quais são as quatro formas principais de sincronização citadas para programas de memória compartilhada?
3. Quando deve ser utilizada a sincronização por variáveis condicionais?
4. Como os semáforos funcionam e quais são as duas operações atômicas utilizadas para acessá-los?
5. Explique a diferença funcional entre as operações SIGNAL e BROADCAST em uma variável de condição.
6. Por que uma variável de condição deve ser sempre usada em conjunto com um mutex?
7. O que acontece com o lock (mutex) quando uma *thread* executa a operação 'WAIT' e o que ocorre quando ela retoma a execução?

-
8. Defina o conceito de sincronização por barreira e explique por que ela é comparada a um "ponto de encontro".
 9. No contexto do Método de Jacobi, por que o uso de barreiras é fundamental para garantir que os resultados de cada iteração sejam avaliados corretamente?
 10. Quais são as rotinas da biblioteca Pthreads utilizadas para inicializar, esperar e destruir uma barreira?
 11. O que é um semáforo em sistemas operacionais?
 12. Qual a diferença entre `'wait()'` e `'signal()'`?
 13. Qual a diferença entre semáforo binário e contador?
 14. Como um semáforo ajuda a evitar condições de corrida?
 15. Em que situações um semáforo é mais útil que um *mutex*?
 16. Quais as principais limitações do semáforo?
 17. Explique a diferença de escopo entre um semáforo inicializado com `sem_init` e um aberto com `sem_open`. Qual deles é mais adequado para sincronizar dois processos independentes que não compartilham memória?
 18. Por que a operação `pthread_cond_wait` precisa liberar o mutex associado de forma atômica ao colocar a thread para dormir? O que aconteceria se houvesse um intervalo de tempo entre a liberação do lock e o bloqueio da thread?
 19. Descreva o problema da inversão de prioridade mencionado como uma limitação dos semáforos. Como isso pode afetar a previsibilidade de um sistema de tempo real?
 20. Tente implementar o comportamento de uma barreira para 3 *threads* utilizando apenas um Mutex e uma Variável de Condição (sem usar `pthread_barrier_t`). A última *thread* a chegar deve dar o sinal para todas as outras.
 21. Com base no exemplo do carro, modifique o código para que existam dois tipos de vagas: "Vagas VIP"(2 vagas) e "Vagas Comuns"(3 vagas). Use dois semáforos diferentes e faça com que alguns carros tentem primeiro a vaga VIP e, se estiver lotada, tentem a comum.
 22. Estenda o exemplo de produtor-consumidor para usar um *array (buffer)* de 5 posições. O produtor só pode produzir se houver espaço, e o consumidor só pode consumir se o *buffer* não estiver vazio. Utilize variáveis de condição para gerenciar esses estados.

Referências

ANDREWS, Gregory R. **Concurrent Programming: Principles and Practice**. [S. l.]: Addison-Wesley, 1991.

ANDREWS, Gregory R. **Foundations of Multithreaded, Parallel, and Distributed Programming**. [S. l.]: Addison-Wesley, 2000.

CARVER, Richard H.; TAI, Kuo-Chung. **Modern Multithreading**. [S. l.]: Wiley, 2006.

HERLIHY, Maurice; SHAVIT, Nir. **The Art of Multiprocessor Programming**. [S. l.]: Morgan Kaufmann, 2008.

SCOTT, Michael L. **Programming Language Pragmatics**. 2. ed. [S. l.]: Morgan Kaufmann, 2006.

SILVA, Gabriel P.; BIANCHINI, Calebe P.; COSTA, Evaldo B. **Programação Paralela e Distribuída com MPI, OpenMP e OpenACC para Computação de Alto Desempenho**. [S. l.]: Casa do Código, 2022.

TAUBENFELD, Gadi. **Synchronization Algorithms and Concurrent Programming**. [S. l.]: Pearson Prentice Hall, 2006.